

Supporting Information for Visualizing Chemical Principles Using Interactive Python Modules

Lindsay Clos[†], Katherine Smith[†], Matthew D. Hanson*

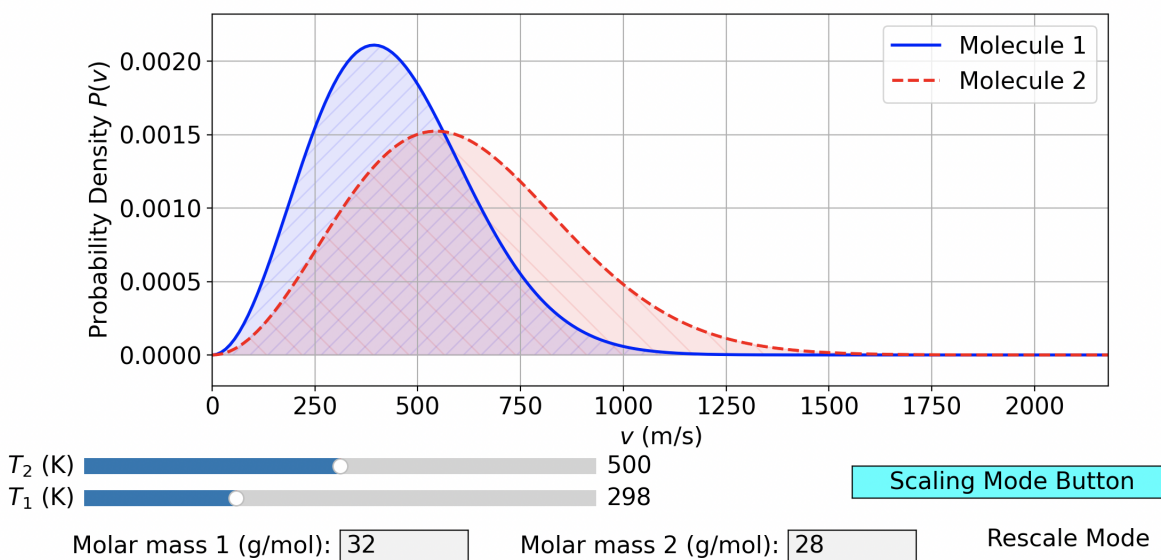
Department of Chemistry, Le Moyne College, Syracuse, NY 13214
Email: hansoma@lemoyne.edu

Contents

1	About This Guide	S2
2	Python and Package Installation	S3
2.1	Quick Installation Guide	S3
2.2	Additional Installation Explanations	S3
3	Running the Programs	S6
4	Program Features, Conventions, and Select Data	S7
4.1	Born-Haber_GUI.py	S7
4.2	Activation Energy Plots	S9
4.3	Kinetic_Order_Plots_GUI.py	S10
4.4	Gibbs Free Energy and Equilibrium Plots	S11
4.5	Acid-Base Titration Plots	S12
4.6	Heating_Curve_GUI.py	S14
4.7	Ideal_Gas_GUI.py and VDW_Gas_GUI.py	S15
4.8	Maxwell-Boltzmann Distributions	S17
4.9	Sphere_Points.py	S18
4.10	Morse Potential Curves	S19

1 About This Guide

This document includes instructions for installing and using the GUI-based Python modules outlined in the main article. Each of these modules is a standalone Python script that needs no separate installation once the necessary Python software has been installed. These modules can be run either using an interpreter like the IDLE interpreter that comes with Python or with a command line command. The features of each of these modules and the conventions for their proper use are also outlined in this document. For convenience, a set of parameters for some common molecules or links to easily accessible data are given to aid instructors with using these tools in the classroom or for generating problems, where appropriate. Note that the codes are designed to be robust so that users inputting physically unreasonable values will result in guidance messages rather than an error whenever possible.



2 Python and Package Installation

2.1 Quick Installation Guide

The following steps should be followed to install the necessary packages locally on a computer:

1. Download and install Python 3. **Windows users should check the box that installs Python3 on the user's PATH during installation!**
2. Install and/or open a terminal application and run the following command to install the necessary Python packages and their dependencies,

```
python3 -m pip install numpy scipy matplotlib
```

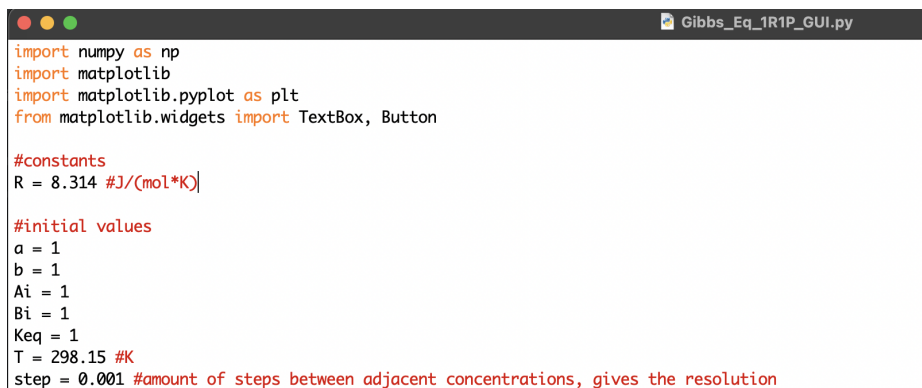
You may be prompted to confirm the downloads of various packages by typing “Y” and Enter/return into the terminal. If this does not run properly, try removing the “3” from the end of the python3 command. If one or more packages fails to install properly it is worth trying to install the package individually using the same installation command with only the offending package(s).

All modules should now be usable on your computer either through an interpreter like IDLE or via command line.

2.2 Additional Installation Explanations

The need for installation instructions comes from the various Python packages used in the Python scripts and their large number of dependencies in Python. Python is an extremely popular and relatively user-friendly coding language. For Windows users, be sure to choose the option to include Python on the user's PATH when first installing it, or else the executable command for Python in the terminal will not actually work! The PATH environment variable is where the computer looks to find executable files for various programs. You can also manually modify the PATH on Windows by searching for the environment variables and adding a new PATH variable corresponding to the locations of executable files on your computer. This can be used if Python is accidentally installed without adding the executable to the PATH, but will not be covered here.

The reason we need to install additional modules is that Python does not have all available features activated unless they are requested via the import command. Some packages with niche technical uses are not installed by default, which necessitates obtaining them from an external source. This allows programs to use fewer computer resources processing unnecessary computer code and saves hard drive space for those that do not need certain packages. In each program, there is a block of code that imports the necessary Python packages at the start of the module, as shown in the code sample below.



```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.widgets import TextBox, Button

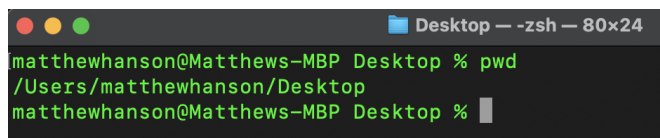
#constants
R = 8.314 #J/(mol*K)

#initial values
a = 1
b = 1
Ai = 1
Bi = 1
Keq = 1
T = 298.15 #K
step = 0.001 #amount of steps between adjacent concentrations, gives the resolution
```

To install the necessary packages on either Windows or MacOS computers, just run the following command in a terminal application once Python has been installed,

```
python3 -m pip install numpy scipy matplotlib
```

On a Mac terminal application the terminal may look like the example below.



```
matthewhanson@Matthews-MBP Desktop % pwd
/Users/matthewhanson/Desktop
matthewhanson@Matthews-MBP Desktop %
```

In this example, the command `pwd` prints the path to the current directory that the terminal is accessing. To find the terminal application, on MacOS simply open a Finder window and navigate to Applications/Utilities/Terminal. It is recommended to make the Terminal application a shortcut on the user's toolbar for ease of use. On Windows, simply type "PowerShell" into the search bar near the Start menu and select the Windows PowerShell application. If PowerShell is not already installed, it can be downloaded for free from the Microsoft store. Once again, it is recommended that the user pin a terminal application to their toolbar for ease of access later. No knowledge of operating the terminal is required to utilize these modules other than following the commands explicitly laid out in these instructions.

The "pip install" command is a built-in command included with Python that is used to install additional packages to the Python distribution. The pip command for Python3 is also frequently used in the form,

```
pip3 install numpy
```

but the version of the command,

```
python3 -m pip install
```

ensures that executable files are installed on the user's PATH for Windows machines. Note that some installations of Python will only recognize the execution command "python" in a terminal, so the above installation command can be modified by removing the "3" from the end of the python3 command. Note that the above commands can be run for individual packages (e.g. "pip3 install numpy") or for some small subset of the packages (e.g. "python3

-m pip install numpy scipy”), if desired, though it is easiest to install many at once. Note that when running pip on its own, the command pip3 must be run for installation of packages into Python3. Use of “pip install” rather than “pip3 install” will attempt to install packages for Python2 for MacOS users, which has now been deprecated despite being installed for MacOS by default.

Some of the necessary packages may already be installed depending on the user’s Python distribution. For instance, the numpy and scipy modules are typically installed in Anaconda distributions of Python. Attempting to install packages that are already installed will not cause problems, but the user will see a large number of messages in the terminal indicating that the package dependencies are already satisfied.

Package managers like Anaconda and Mamba, which can be downloaded by following the relevant hyperlinks, are also capable of running Python in a virtual environment. This circumvents the need for local installations of all the requisite packages. This can be important for particular computer environments. To create a virtual environment called “pymods” with the necessary Python packages in Mamba, use the following commands in a terminal,

```
mamba create -n pymods python numpy scipy matplotlib
mamba activate pymods
python Module_Name.py
```

where `Module_Name` should be replaced with the desired module name. When you are finished with the program, the following terminal commands can be used to terminate the virtual environment,

```
mamba deactivate
mamba remove -n pymods --all
```

The same process of creating a virtual environment can be achieved in Anaconda by using the terminal commands,

```
conda config --add channels conda-forge
conda config --set channel_priority strict
conda create -n pymods python numpy scipy matplotlib
conda activate pymods
python Module_Name.py
```

where again `Module_Name` must be replaced with the name of the desired module. The virtual environment can be deactivated using the terminal commands,

```
conda deactivate
conda remove -n pymods --all
```

3 Running the Programs

To run the Python script files when the `Chemistry_Python_Modules` folder is placed on the user's desktop, the following steps can be used:

1. Open the terminal application and type the following commands sequentially,

```
cd Desktop/Chemistry_Python_Modules
python3 Module_Name.py
```

where the `Module_Name` is the name of the desired module. If the latter fails, try removing the “3” from the `python3` command.

2. Utilize the interactive GUI elements to update or create plots as desired. The window for the program can be closed to end the program.

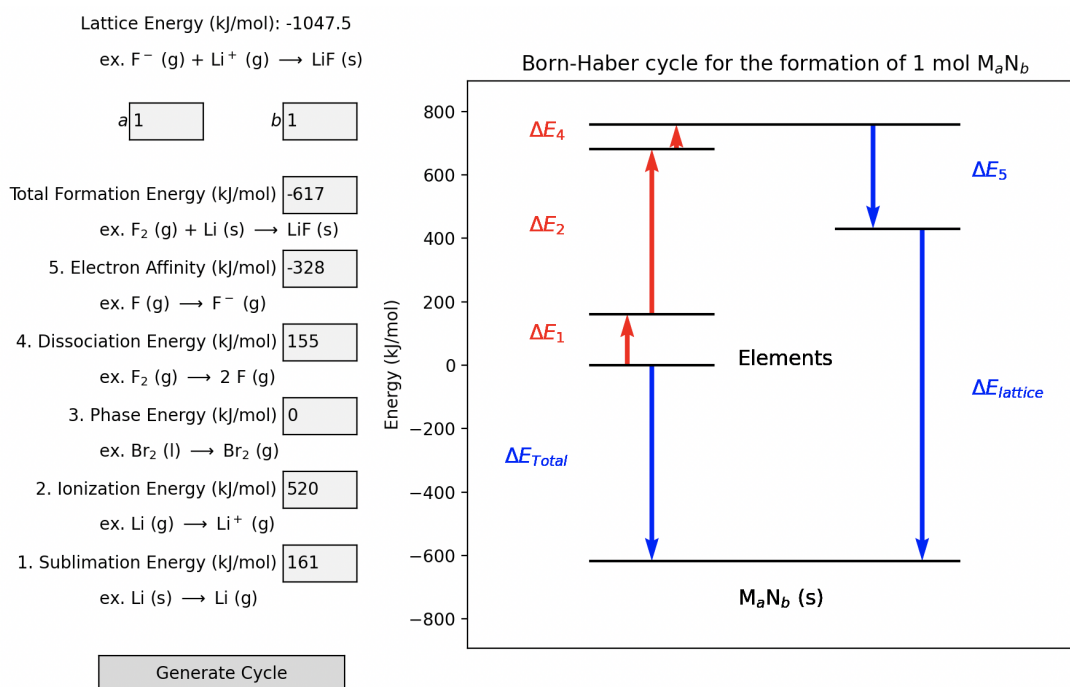
Alternatively, the module file can be opened in an editor like the IDLE interpreter that comes with Python and run through the interpreter's shell. The module can be run in IDLE by pressing the F5 key while in the module file. Note that because of imperfections in the Python launcher the widgets can sometimes become “stuck.” The best solution is to simply close the window and re-run the code after a few seconds.

4 Program Features, Conventions, and Select Data

This section will outline the features of the various Python modules by topic. Modules that are functionally similar will be grouped together. These topics are in no particular order due to their variety. The chemical conventions of each program will also be stated, when necessary.

4.1 Born-Haber_GUI.py

The first module we will introduce is one of the most complex. The Born-Haber_GUI.py module allows the automatic generation of plots that show the Born-Haber cycle for the formation of an ionic solid of the form M_aN_b , where M is a metal atom and N is a non-metal atom, from its constituent elements in their natural state, as shown in the image below.



The program's primary usage is showing how the lattice energy varies for various ionic solids; the lattice energy displayed at the top of the user inputs automatically adjusts whenever the button is pressed. The program also generates a graphic with the full Born-Haber cycle, including arrows that are generally colored based on the sign of the energy involved in that step of the process. The program takes in the energy (or enthalpy—the plot will just say energy on the y axis and these values are usually almost the same) required for each of several reaction steps:

1. The sublimation (or vaporization, in the case of mercury) energy of the metal element. This is ΔE_1 on the graph.
2. The total ionization energy of the metal atom. For cations of higher charge, all of the relevant ionization energies must be added together. Ionization energy data are readily available on the excellent Ptable.com. This is ΔE_2 on the graph.

3. The energy of any phase transitions necessary to bring the non-metal element in their natural state into the gas phase. If the element is already naturally in the gas phase, this value can be left as 0 and the corresponding step will be omitted from the produced graphic. This would be ΔE_3 if it were shown on the graph.
4. The dissociation energy of the non-metal element, if any. If this energy is left as 0 the relevant step in the cycle will also be omitted. Note that the reaction stoichiometry assumes that one mole of the diatomic is broken up, producing a pair of atoms. Thus, only half of this energy is accounted for in the calculation of the lattice energy! This is double ΔE_4 on the graph.
5. The electron affinity of the non-metal atom. These are typically negative by convention in this program. Electron affinity data for adding a single electron are readily available on Ptable.com, though they are listed with typically positive values by convention on that site. This is ΔE_5 on the graph.
6. The overall energy of formation for the ionic solid from its constituent elements. This is ΔE_{Total} on the graph.

The program also takes into account the stoichiometry of the solid with the coefficients a and b in determining the lattice energy with the formula,

$$\Delta E_{lattice} = \Delta E_{Total} - a\Delta E_1 - a\Delta E_2 - b\Delta E_3 - \frac{b}{2}\Delta E_4 - b\Delta E_5$$

This allows for the quantitative analysis of solids like alkali metal oxides or alkali earth halides with no issues. The default values that the program uses are for the formation of the solid LiF. Some values for a small selection of solids with various cations and anions are given in the table below. In this table, each energy corresponding to the energy for the appropriate element or ion necessary to make the solid (all ΔE values in kJ/mol) and the ionization and electron affinity data has been summed for the necessary charges. The lattice energy $\Delta E_{lattice}$ calculated from the program is given in the final column, all of which agree reasonably well with accepted values.

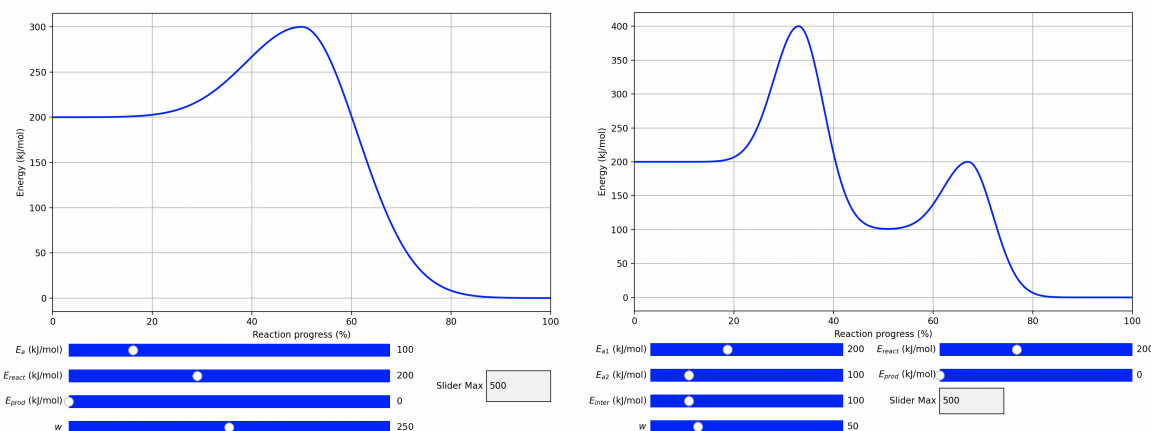
Solid	ΔE_1	ΔE_2	ΔE_3	ΔE_4	ΔE_5	ΔE_{Total}	$\Delta E_{lattice}$
LiF	161	520	0	155	-328	-617	-1047.5
LiCl	161	520	0	243	-349	-409	-862.5
NaCl	107	496	0	243	-349	-411	-786.5
KCl	89	419	0	243	-349	-438	-718.5
KI	89	419	57	151	-295	-328	-673.5
Na ₂ O	107	496	0	495	603	-416	-2472.5
CaO	178	1735	0	495	603	-603	-3366.5
CaBr ₂	178	1735	31	193	-325	-675	-2193

4.2 Activation Energy Plots

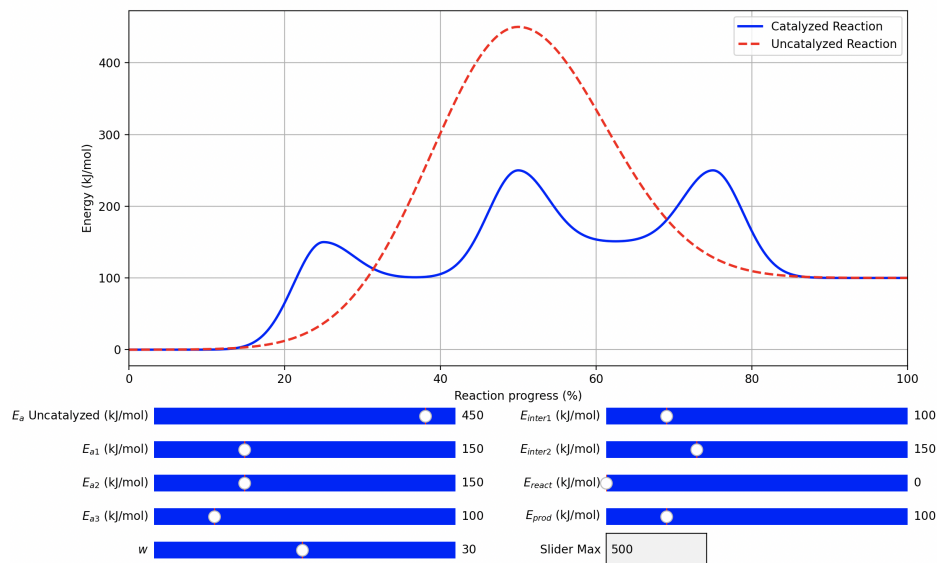
The following set of modules allow for the generation of graphs that qualitatively illustrate the concept of activation energy for reactions that can be endoergic or exoergic with or without the presence of reaction intermediates,

- Ea_plot_1peak.py
- Ea_plot_1peak_catalyst.py
- Ea_plot_2peak.py
- Ea_plot_2peak_catalyst.py
- Ea_plot_3peak.py
- Ea_plot_3peak_catalyst.py

Because the energies involved in reaction mechanisms are generally quite complex, this is done qualitatively by stitching together Gaussian (bell curve) functions with heights controlled by the relevant activation energies, reactant and product energies, and intermediate energies (where appropriate). The width of the Gaussians is controlled by a parameter w . This is shown in the examples of single and two step exothermic processes without catalysts below.



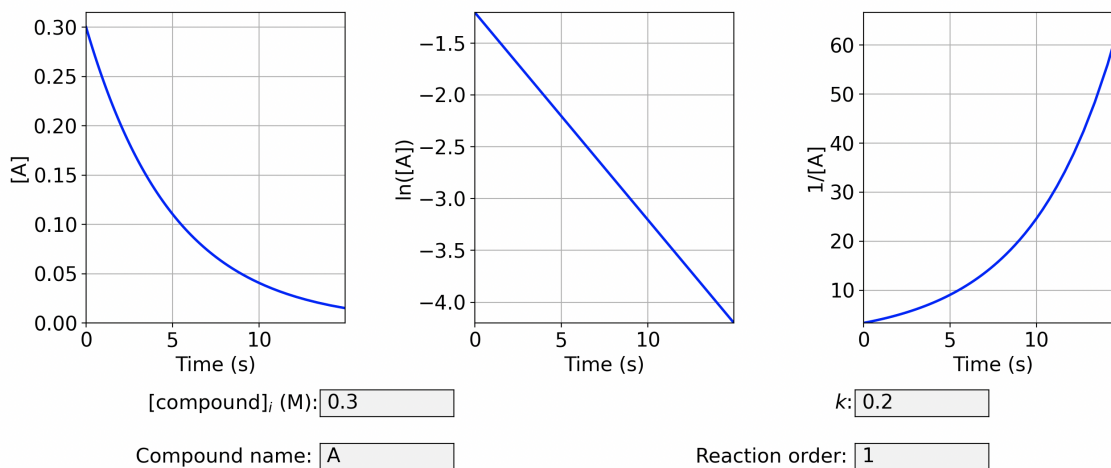
These modules utilize sliders to allow users to dynamically update the graphs in real time. The sliders have built-in safety checks to ensure that the values do not result in physically nonsensical situations (i.e. the activation energy being insufficient to reach the next step of the reaction). The sliders are limited to the value set in the Slider Max box, which can be adjusted to give a broader or smaller range as needed. The modules that include “catalyst” in the name can be used to illustrate how a catalyst might change the mechanism of a reaction by lowering the activation energy and potentially adding additional steps. For simplicity, these modules only allow for a single-step uncatalyzed reaction, as shown below for an endothermic process.



The activation energy of the uncatalyzed process will automatically adjust if necessary to ensure that the catalyzed process is at least as fast as the uncatalyzed process.

4.3 Kinetic_Order_Plots_GUI.py

This module allows for the generation of plots that can be used to test students' understanding of simple integrated rate laws and how visual data can be used to determine the order of reaction quite easily. As shown below, the plots of $[A]$, $\ln([A])$, and $1/[A]$ vs. time can be used to determine the order of reaction since these would correspond to linear plots for 0th, 1st, or 2nd order single-component reactions, respectively.



The boxes also allow for putting a name, with LaTeX conventions used for subscripts and superscripts (i.e. CO_3^{2-}) as well as concentrations and rate constants. The units of the rate constant are omitted because they vary with the order of reaction.

4.4 Gibbs Free Energy and Equilibrium Plots

The set of Python modules,

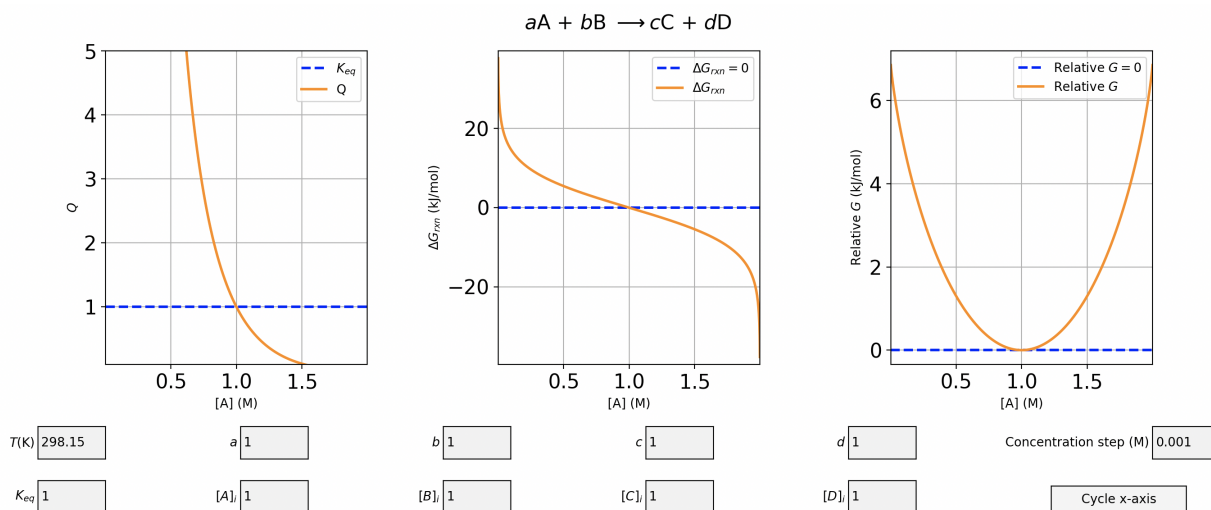
- Gibbs_Eq_1R1P_GUI.py
- Gibbs_Eq_1R2P_GUI.py
- Gibbs_Eq_2R1P_GUI.py
- Gibbs_Eq_2R2P_GUI.py

utilizes the connection between thermochemistry and equilibrium in the equations,

$$\Delta G_{rxn}^{\circ} = -RT \ln(K_{eq})$$

$$\Delta G_{rxn} = \Delta G_{rxn}^{\circ} + RT \ln(Q)$$

to illustrate how different values of the initial concentrations of species with various stoichiometric coefficients give rise to different equilibrium concentrations of these species at a certain temperature. These modules differ only in the number of reactants in products in the reaction shown above the graphs, which allows instructors flexibility in their choices of examples. The example below is for a two reactant and two product reaction.



Each module shows how equilibrium can be considered established in one of three ways:

1. The reaction quotient Q is the same as the equilibrium constant K_{eq} .
2. The $\Delta G_{rxn} = 0$.
3. The relative value of G for all of the reactants and products present has reached a minimum.

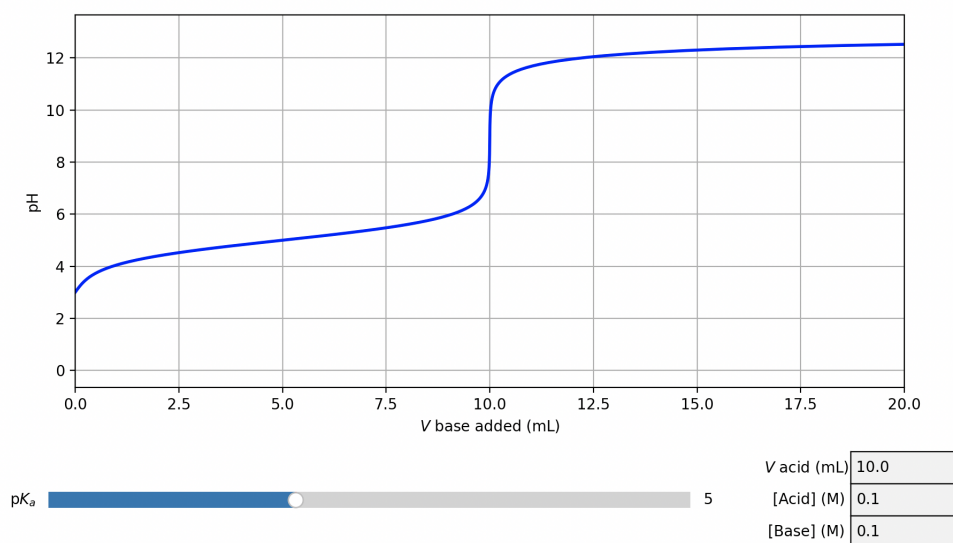
These modules allow both the qualitative and quantitative presentation of complicated relationships between thermodynamic values and the ultimate equilibrium that is established.

4.5 Acid-Base Titration Plots

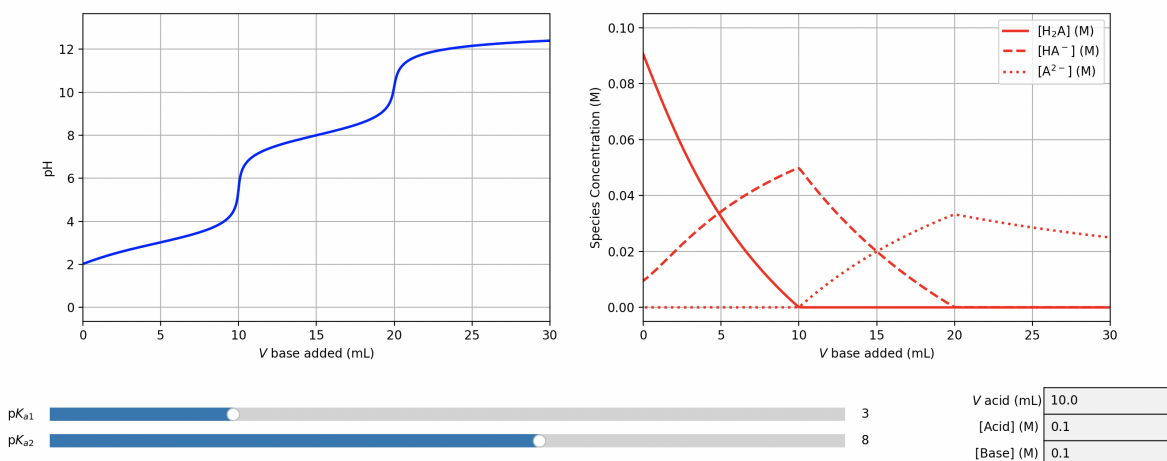
The set of Python modules,

- HA_acid_titration.py
- H2A_acid_titration.py
- H3A_acid_titration.py
- HA_acid_titration_speciation.py
- H2A_acid_titration_speciation.py
- H3A_acid_titration_speciation.py
- HA_acid_titration_derivatives.py
- H2A_acid_titration_derivatives.py
- H3A_acid_titration_derivatives.py

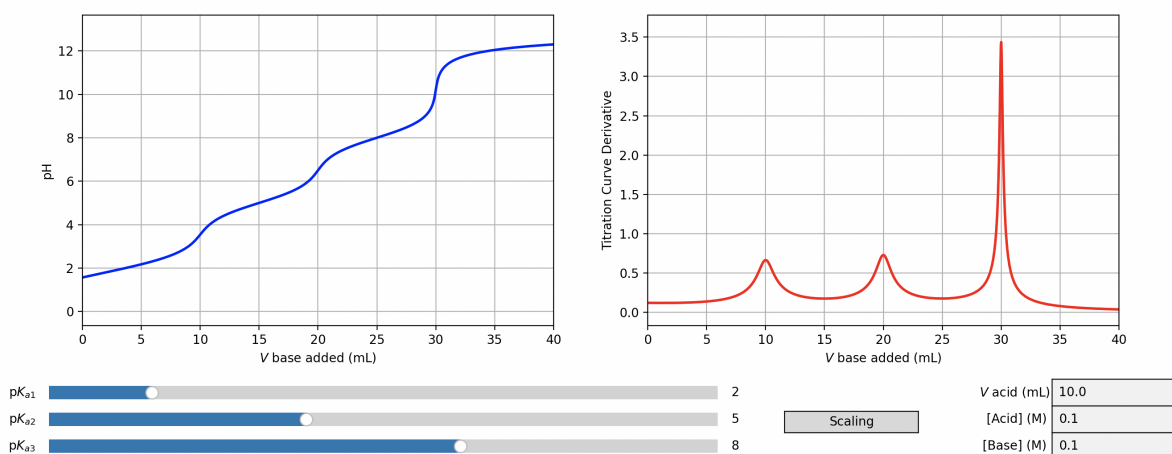
can be used to generate the titration curves of mono- di- and tri-protic acids with a strong base. The first three modules merely create a titration curve with the user controlling the pK_a value(s) by a slider and with the concentration of the acid and base solutions and volume of the original acid sample controlled by boxes, as shown in the example below.



The modules that include speciation in their names additionally create a second subplot that shows the concentrations of all forms of the acid species in solution at every point along the titration curve, as shown in the example below.



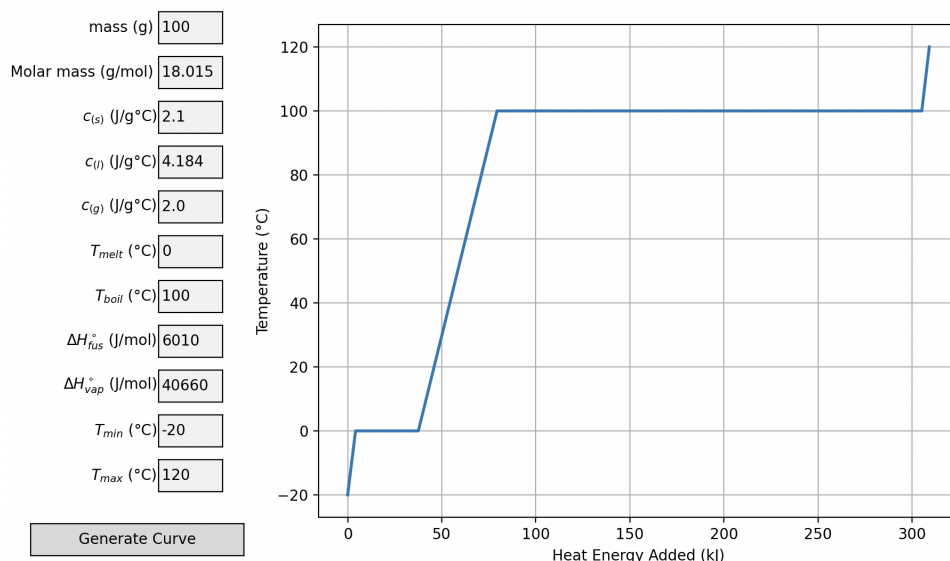
The equations used for the concentration of each species are derived using the conservation of the total amount of the acid species in solution and the Henderson-Hasselbach equation. Note that the dilution of the acid is taken into account in the calculation of the concentrations. The modules with derivatives in their names instead create a second subplot that shows the slope of the titration curve at every point along the curve, as shown in the example below.



The derivative plot is useful for spotting subtle endpoints of polyprotic acids. The scaling button switches the graph between scaling to the highest peak (default) or the lowest peak, which can be useful for identifying subtle endpoints.

4.6 Heating_Curve_GUI.py

The Heating_Curve_GUI.py module can be used to generate heating curves for substances. This module takes into account the various heat capacities of each phase of the substance, the phase transition temperatures, the enthalpies of each of these phase transitions, the amount of substance, the molar mass, and allows the user to specify the temperature range of interest. This is shown in the example below.



The program will also read off precise values of the heat involved in the temperature changing regions and any phase transitions, as shown below.

```

Python 3.13.3 (v3.13.3:6280bb54784, Apr  8 2025, 10:47:54) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/matthewhanson/Desktop/Chemistry_Python_Modules/Heating_Curve_GUI.py
Heating solid: 4200.0 J
Heating liquid: 41840.0 J
Heating vapor: 4000.0 J
Melting: 33361.09 J
Boiling: 225700.8 J

>>>

```

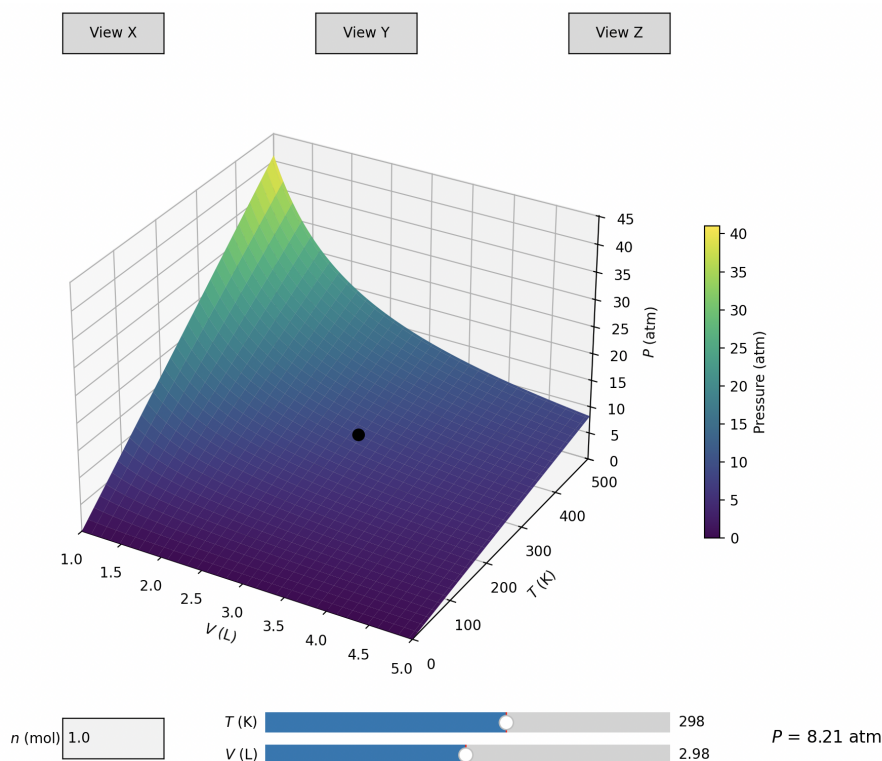
The isobaric specific heat capacities (in J/g°C), melting and boiling points (°C), standard enthalpy of fusion and vaporization (J/mol), and molar masses of a few select substances are given in the table below for convenience.

Substance	Molar mass	$c_{(s)}$	$c_{(l)}$	$c_{(g)}$	T_{melt}	T_{boil}	ΔH_{fus}°	ΔH_{vap}°
H ₂ O	18.015	2.1	4.184	2.0	0	100	6010	40660
NH ₃	17.03	2.1	4.7	2.16	-77.7	-33.3	5650	23500
Cu	63.546	0.385	0.572	0.327	1085	2562	205	300300
CH ₃ CH ₂ OH	46.069	0.97	2.50	1.41	-114.1	78.4	46000	42300
C (graphite)	12.011	0.72	0*	0.327	3650*	3650	0*	717000

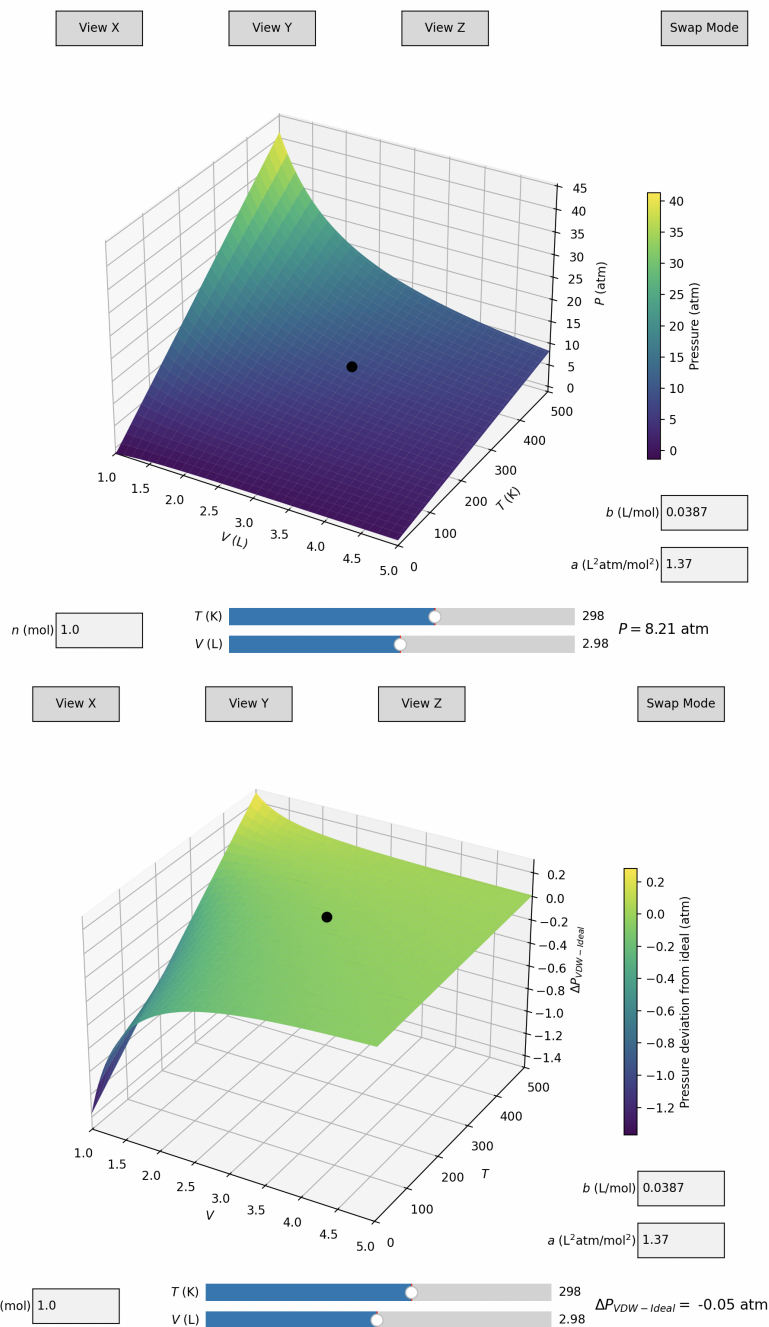
*Note that graphite does not melt, it directly sublimates to the gas phase. Thus, we listed the heat capacity of the liquid phase as 0, the enthalpy of fusion as 0, and the melting point as the same as the boiling point so the calculations will come out correctly.

4.7 Ideal_Gas_GUI.py and VDW_Gas_GUI.py

The Ideal_Gas_GUI.py module creates a 3-D surface plot that displays the relationship between V , T , and P in the ideal gas law for a fixed amount n of gas according to the ideal gas law. It also uses sliders to allow the user to move a point around on this surface to see how the pressure varies with changing pressure and temperature more easily. The moles of gas can be adjusted using the dedicated text box. Because 3-D plots can be somewhat awkward to maneuver in Python, dedicated buttons that orient the viewing window along the x , y , and z axes are added at the top of the window. The pressure at the point that moves about the surface is displayed on the bottom right, as shown below.



The VDW_Gas_GUI.py module creates a 3-D surface plot of how the pressure of a Van Der Waals (VDW) gas varies with both the temperature and volume. The produce plots look highly similar to the ideal gas plot for typical values of the Van Der Waals parameters a (the pressure correcting term) and b (the volume correcting term). To facilitate identifying the differences, an additional button is added on the top right that switches to showing the deviation of the VDW gas pressure from the ideal gas. A negative deviation means that the VDW gas has a lower pressure. The same functionality of moving a point about on the surface exists in both modes. Two new boxes are added for changing the parameters a and b . Both modes are shown below.



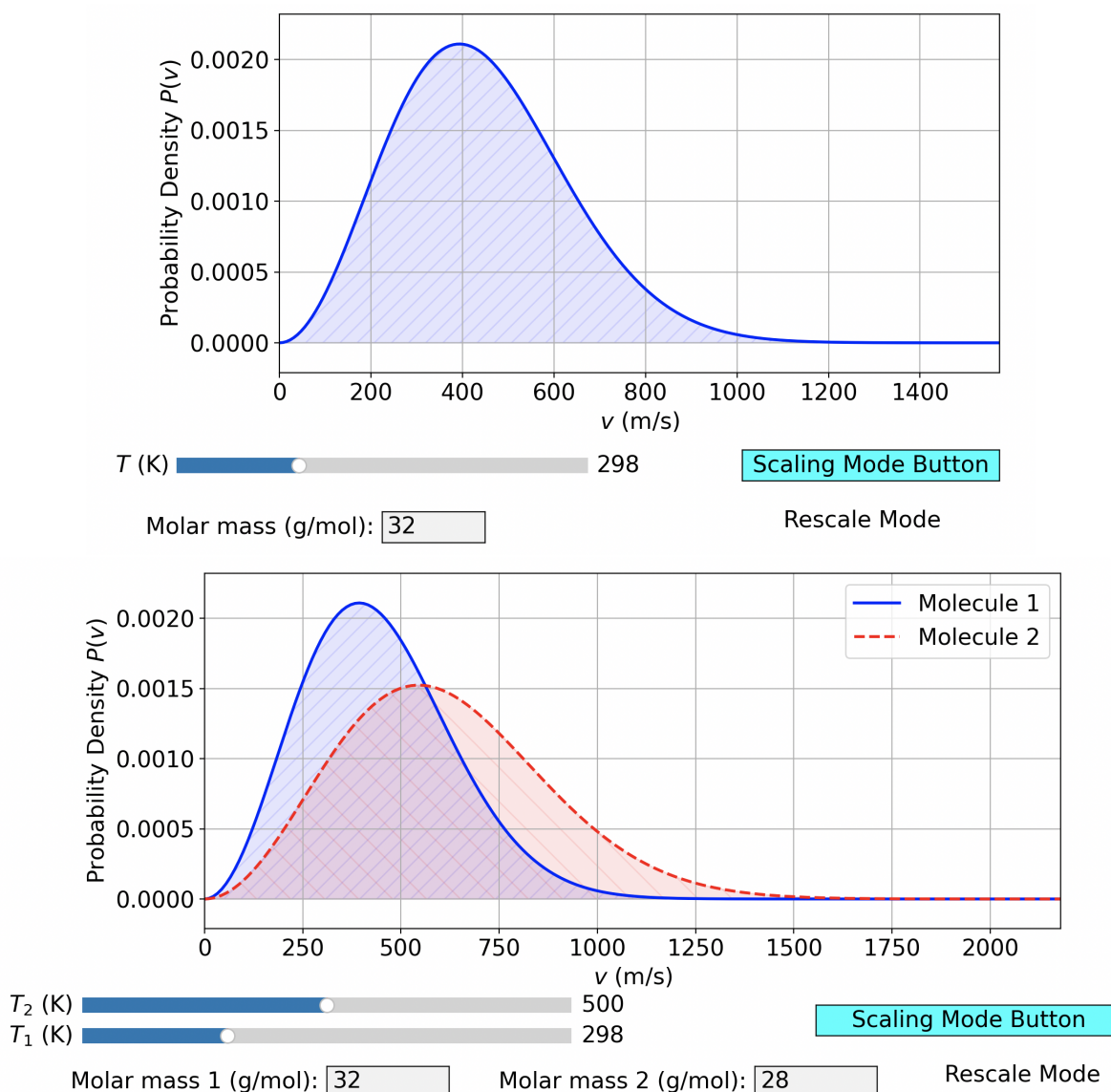
The Van Der Waals parameters for a few select substances are given below for convenience.

Gas	a ($L^2 atm/mol^2$)	b (L/mol)
Ne	0.214	0.0171
N ₂	1.370	0.0387
O ₂	1.382	0.0319
CO ₂	3.60	0.0427
N ₂ H ₄	8.46	0.0462

Note that negative gas pressures are a specialized artifact of the VDW equation.

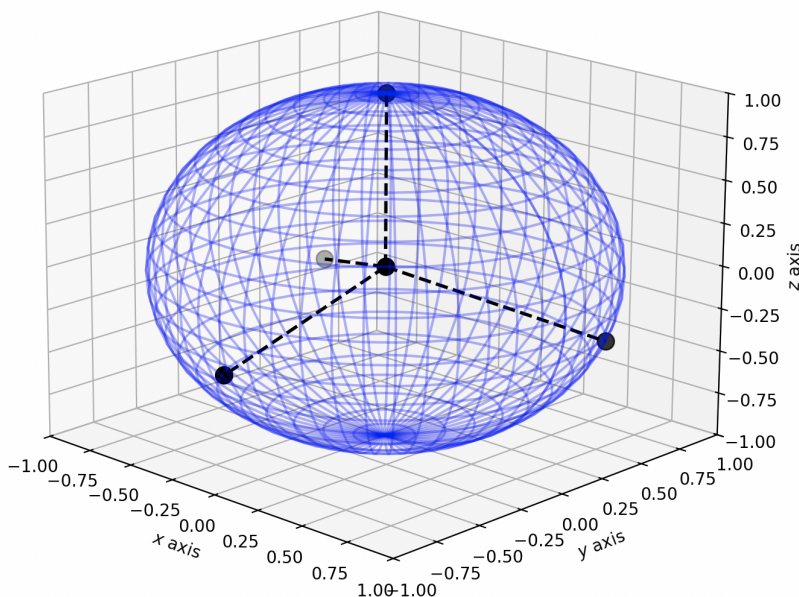
4.8 Maxwell-Boltzmann Distributions

The `Maxwell-Boltzmann_Single_GUI.py` and `Maxwell-Boltzmann_Double_GUI.py` modules both allow the user to create Maxwell-Boltzmann distributions of the translational speeds of gas molecules based on the molar mass of the gas and the temperature. There is also a button to change the axes from constantly rescaling to retain the aspect ratio or fixing the current axes and just updating the curve. This allows for more dynamic control of the visuals and different emphases for teaching. Note that otherwise these modules only differ in that `Maxwell-Boltzmann_Double_GUI.py` allows the simultaneous comparison of two molecules' speed distributions at the same time, as shown below.



4.9 Sphere_Points.py

This module allows the user to create a plot of a sphere with the number of requested points plotted as far apart from one another as possible by giving an integer in response to a prompt, as shown below.

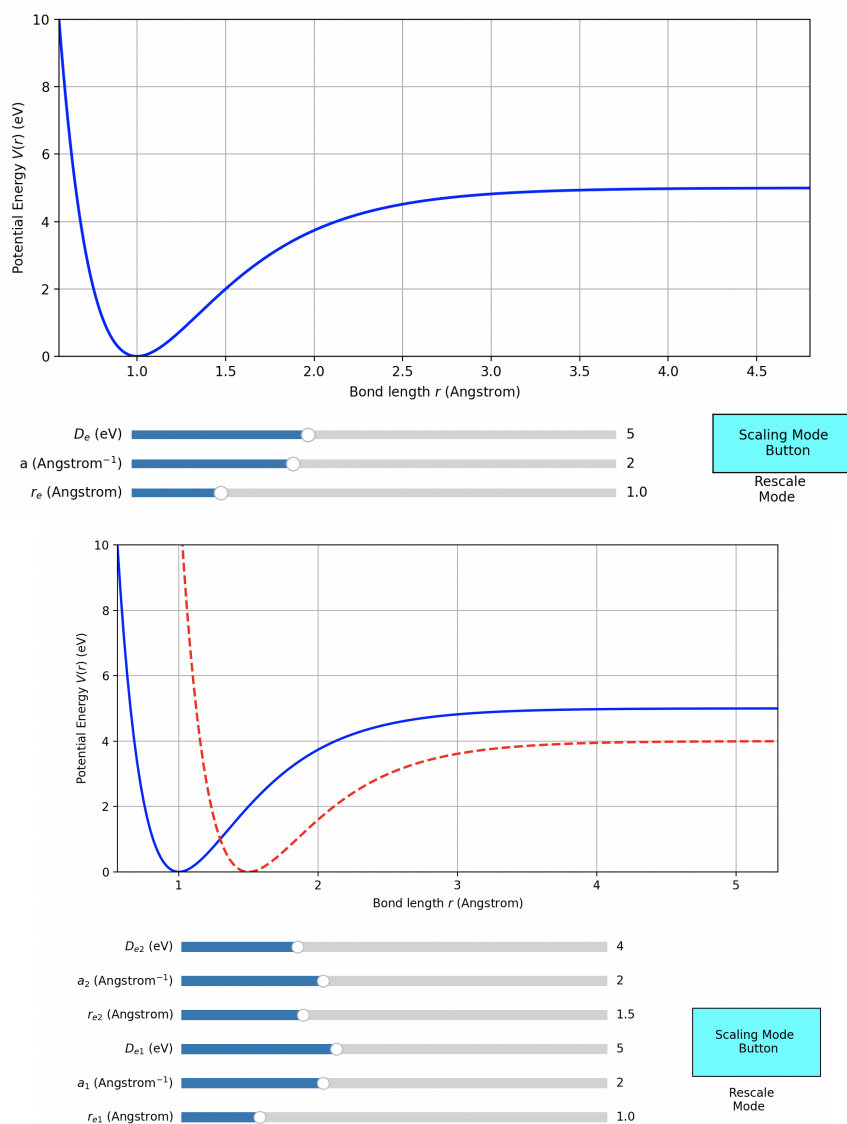


Note that going above 7 points takes a very long time because the difficulty of the nonlinear optimization scales exponentially with the number of independent points (and is chemically outrageous anyway)! The point locations, the angles between them, and some modest data about the optimization cycles are printed to the console, allowing students to see where the ideal geometric angles in molecules come from, as shown below.

```
IDLE Shell 3.13.3
Python 3.13.3 (v3.13.3:6280bb54784, Apr 8 2025, 10:47:54) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/matthewhanson/Desktop/Chemistry_Python_Modules/Sphere_points.py
How many points should be plotted? 4
Optimization converged: True
Number of iterations required: 246
Coordinates of optimized points:
[[ 0.  0.  1. ]
 [ 0. -0.943 -0.333]
 [-0.816  0.471 -0.333]
 [ 0.816  0.471 -0.333]]
Angles between points:
[[ 1.  0. 109.5]
 [ 2.  0. 109.5]
 [ 2.  1. 109.5]
 [ 3.  0. 109.5]
 [ 3.  1. 109.5]
 [ 3.  2. 109.5]]
>>>
```

4.10 Morse Potential Curves

Both the Morse_Potential_Single_GUI.py and Morse_Potential_Double_GUI.py modules allow the user to create Morse potential curves that show bond dissociation energies (D_e), stable equilibrium bond lengths (r_e), and slight differences in potential shape (determined by a) for various molecules. The only difference between these modules is that in the Morse_Potential_Double_GUI.py there are two Morse potential curves that allow direct comparison of the bonding curves of two molecules simultaneously, as shown below.



Parameters for select diatomic molecules are given in the table below for convenience.

Molecule	D_e (eV)	a (Å ⁻¹)	r_e (Å)
I ₂	1.54	1.80	2.666
H ₂	4.75	1.94	0.741
HCl	4.43	1.87	1.274
NO	6.50	2.50	1.151
CO	11.09	2.63	1.128